

1 / 3 · 시계열 데이터 준비

과거 T개를 입력으로, 바로 다음 값을 정답으로 만들어 loader에 넣는다.

실행: Open 열 → 시간 분할 → scaling → window → Tensor → loader | 역추적: batch ← X,y ← window

[범위 핵심] 시간 분할과 MinMaxScaler

```
training_data_len = math.ceil(len(df) * train_ratio)
train_data = df[:training_data_len][["Open"]]
test_data = df[training_data_len:][["Open"]]
scaler = MinMaxScaler(feature_range=(0, 1))
train_scaled = scaler.fit_transform(train_data.values)
test_scaled = scaler.transform(test_data.values)
```

인자 출처 df: 다운로드한 시계열 DataFrame. train_ratio: 앞선 셀 설정(원본 0.8). Open: 선택한 특성 열. .values: [날짜 수, 1] 배열. scaler는 바로 위 객체.

실수 방지 train에서만 fit하여 최소-최대 기준을 학습합니다. test는 transform만. test가 train 범위를 넘으면 0~1 밖도 가능합니다. 시계열의 시간 분할과 학습 window 배치의 shuffle=True는 다른 단계입니다.

근거: ts_solution.ipynb · 원본 셀 12,14

[기출 복기] Sliding window → Tensor → DataLoader

```
# sequence_length = 50: 앞선 셀의 설정
def convert_data_into_tensors(data_seq):
    features, labels = [], []
    for i in range(len(data_seq) - sequence_length):
        features.append(data_seq[i:i + sequence_length])
        labels.append(data_seq[i + sequence_length, 0])
    features = torch.tensor(np.array(features), dtype=torch.float32)
    labels = torch.tensor(np.array(labels), dtype=torch.float32)
    return features, labels

def to_loader(x, y, batch_size, shuffle):
    dataset = torch.utils.data.TensorDataset(x, y)
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle)
```

인자 출처 data_seq: train_scaled/test_scaled [N,1]. sequence_length: 함수 밖 설정. i: for 반복 인덱스. label의 0: 단일 특성 Open의 열 인덱스. x,y,batch_size,shuffle: to_loader 입력.

실수 방지 data=[10,20,30,40,50], T=3이면 X=[10,20,30], y=40. 샘플 수=N-T. X [N-T,T,1], y [N-T]; 배치 [B,T,1], [B]. 정답은 마지막 입력 i+T-1이 아니라 그 다음 i+T.

근거: ts_solution.ipynb · 원본 셀 16,18

가리고 쓰기: fit_transform/transform 구분, features 슬라이스, labels 인덱스, dtype, TensorDataset 인자.

2 / 3 · RNN/LSTM·Conv1D와 학습

모델의 출력 전체에서 마지막 시점 하나를 골라 회귀 손실과 연결한다.

실행: [B,T,F] → 모델 → [B,T,1] → [B] 예측 → MSE | 역추적: loss ← 예측 [B] + y [B]

[기출 복기 + 범위 핵심] 모델 생성과 forward

```
# __init__(input_size, hidden_size, num_layers)
self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                    batch_first=True)
self.linear = nn.Linear(hidden_size, 1)
# LSTMModel.forward(self, x)
out, _ = self.lstm(x)
return self.linear(out)

# Conv1DModel.__init__(input_size, hidden_size)
self.conv1d = nn.Conv1d(input_size, hidden_size, kernel_size=2, stride=1)
self.fc = nn.Linear(hidden_size, 1)
# Conv1DModel.forward(self, x)
x = x.transpose(1, 2)
x = self.conv1d(x)
x = x.transpose(1, 2)
return self.fc(x)
```

인자 출처 input_size=X_train.shape[-1] (Open 1개). hidden_size·num_layers: 설정값(64,2). x: loader 입력 [B,T,F]. fc/linear의 입력 차원은 직전 hidden 채널 수.

실수 방지 LSTM: [B,T,F]→[B,T,H]→[B,T,1]. RNN/GRU도 batch_first 인자와 출력 연결은 유사. Conv1D: [B,T,F]→[B,F,T]→[B,H,T-1]→[B,T-1,H]→[B,T-1,1]. 이 원본 forward에는 flatten이 없습니다.

근거: ts_solution.ipynb · 원본 셀 22,30,35

[범위 핵심] 마지막 시점·학습·평가

```
loss_fn = nn.MSELoss(reduction="mean")
optimizer = optim.Adam(model.parameters(), lr=1e-3)
pred = model(batch_x[:, -1, 0])
loss = loss_fn(pred, batch_y)
optimizer.zero_grad()
loss.backward()
optimizer.step()

model.eval()
with torch.no_grad():
    test_predictions = model(X_test[:, -1, 0])
test_predictions = test_predictions.cpu().numpy()
y_test = y_test.cpu().numpy()
rmse = root_mean_squared_error(y_test, test_predictions)
```

인자 출처 model: 생성한 모델. batch_x, batch_y: train_loader의 배치. X_test: 테스트 Tensor. metric 호출의 y_test는 CPU numpy로 바꾼 정답. lr은 문제/설정값.

실수 방지[:, -1, 0]은 모든 배치·마지막 시간·출력 특성 0번 → [B]. 예측 [B,1]과 y [B]를 그대로 MSE에 넣으면 잘못 broadcast될 수 있습니다. RMSE=sqrt(MSE), MAE=평균 절댓값 오차. 원래 가격 단위 평가라면 양쪽 모두 역스케일.

근거: ts_solution.ipynb · 원본 셀 24,28; metric 설명은 공식 범위 보장

가리고 쓰기: nn.LSTM 세 인자의 출처, Conv1D의 두 transpose, pred 슬라이싱, backward→step.

3 / 3 · 추천: NGCF, NCF, 순위 평가

ID를 벡터로 바꾸고, 관측한 아이템의 점수가 더 커지도록 학습한다.

실행: ID/edge → embedding → message → score → BPR | 역추적: 추천 순위 ← score ← user/item 벡터

[범위 핵심] Graph의 ID와 메시지

```
# create_edge_index: user 뒤에 item 노드 배치
dst.append(row["movieId"] + num_users)
# NGCFLayer.forward: edge_index에서 두 끝점 추출
src, dst = edge_index
norm = 1.0 / torch.sqrt(deg[src] * deg[dst])
src_feat = node_features[src]
dst_feat = node_features[dst]
edge_messages_for_dst = self.W1(src_feat) + self.W2(src_feat * dst_feat)
edge_messages_for_dst *= norm.unsqueeze(1)
# NGCF.forward: 각 layer의 표현을 특징 축으로 연결
final_features = torch.concat(layer_outputs, dim=-1)
user_features = final_features[:self.num_users]
item_features = final_features[self.num_users:]
```

인자 출처 row: ratings의 한 행. num_users: 사용자 수. edge_index: [2,E] long. deg: 앞선 index_add_로 집계한 노드 연결 수. W1/W2: 생성자 Linear. layer_outputs: 초기+각 graph layer 결과 목록.

실수 방지 norm [E]를 [E,1]로 바꿔 feature [E,D]에 곱합니다. index_add는 반환값을 받아야 하고 index_add_는 원본 변경. solution의 한 줄에 index_add 반환 누락이 있어 그 오기는 암기하지 않습니다.

근거: RecSys_GCF_sol.ipynb · 셀 10,14,15

[범위 핵심] BPR-NCF-Top-K

```
# bpr_loss(user_emb, pos_item_emb, neg_item_emb, ...)
pos_scores = torch.sum(user_emb * pos_item_emb, dim=1)
neg_scores = torch.sum(user_emb * neg_item_emb, dim=1)
loss = -torch.mean(F.logsigmoid(pos_scores - neg_scores))
# NCF.forward: 두 ID를 별도 Embedding에 넣어 [B,32]씩 조회
input_embedding = torch.cat([user_embedding, movie_embedding], dim=1)
# [B,64] → fc1 → ReLU → fc2 → 평점 [B,1]
topk_scores, topk_indices = torch.topk(scores, k=k)
ndcg_u = dcg / idcg if idcg > 0 else 0.0
```

인자 출처 세 emb: 함수 입력, 모두 [B,D]. scores: 한 사용자의 전체 아이템 점수. k: 평가 함수 입력. dcg/idcg: 앞선 순위 누적값. NCF의 users/movies는 long ID; n_users/n_movies는 생성자 입력.

실수 방지 BPR은 positive-negative의 순서. NCF 원본은 평점 회귀(MSE 계열); 클릭/비클릭 0/1 logit 문제일 때 BCEWithLogitsLoss. Hit@K=정답 하나라도 상위 K에 있다; NDCG는 높은 순위에 더 큰 보상. 원본 NGCF는 Recall-Precision-NDCG 평가.

근거: RecSys_GCF_sol.ipynb · 셀 15,17 / RecSys_NCF.ipynb · Neural_Collaborative_Filtering

가리고 쓰기: item ID offset / norm.unsqueeze(1) / feature concat 축 / BPR 차이 방향 / topk 인자.